

The learning curves of both models on **T1** (Figure 3a) show the benefit of combined symbolic and subsymbolic reasoning: the DeepProbLog model uses the encoded knowledge to reach a higher F1 score than the CNN, and does so after a few thousand iterations, while the CNN converges much slower. We also tested an alternative for the neural network baseline. It evaluates convolutional layers with shared parameters on each image separately, instead of a single set of convolutional layers on the concatenation of both images. It converges quicker and achieves a higher final accuracy than the other baseline, but is still slower and less accurate than the DeepProbLog model. Figure 3b shows the learning curve for **T2**. DeepProbLog achieves a somewhat lower accuracy compared to the single digit problem due to the compounding effect of the error, but the model generalizes well. The CNN does not generalize to this variable-length problem setting.

Program Induction The second set of problems demonstrates that DeepProbLog can perform program induction. We follow the program sketch setting of differentiable Forth [Bošnjak et al., 2017], where holes in given programs need to be filled by neural networks trained on input-output examples for the entire program. As in their work, we consider three tasks: addition, sorting [Reed and de Freitas, 2016] and word algebra problems (WAPs) [Roy and Roth, 2015].

- T3:** `forth_addition/4`: where the input consists of two numbers and a carry, with the output being the sum of the numbers and the new carry. The program specifies the basic addition algorithm in which we go from right to left over all digits, calculating the sum of two digits and taking the carry over to the next pair. The hole in this program corresponds to calculating the resulting digit (`result/4`) and carry (`carry/4`), given two digits and the previous carry.
- T4:** `sort/2`: The input consists of a list of numbers, and the output is the sorted list. The program implements bubble sort, but leaves open what to do on each step in a bubble (i.e. whether to swap or not, `swap/2`).
- T5:** `wap/2`: The input to the WAPs consists of a natural language sentence describing a simple mathematical problem, and the output is the solution to this question. These WAPs always contain three numbers and are solved by chaining 4 steps: permuting the three numbers (`permute/2`), applying an operation on the first two numbers (addition, subtraction or product `operation_1/2`), potentially swapping the intermediate result and the last digit (`swap/2`), and performing a last operation (`operation_2/2`). The hole in the program is in deciding which of the alternatives should happen on each step.

DeepProbLog achieves 100% on the Forth addition (**T3**) and sorting (**T4**) problems (Table 1a). The sorting problem yields a more interesting comparison: differentiable Forth achieves a 100% accuracy with a training length of 2 and 3, but performs poorly on a training length of 4; DeepProbLog generalizes well to larger lengths. As shown in Table 1b, DeepProbLog runs faster and scales better with increasing training length, while differentiable Forth has issues due to computational complexity with larger lengths, as mentioned in the paper.

On the WAPs (**T5**), DeepProbLog reaches an accuracy between 96% and 97%, similar to Bošnjak et al. [2017] (96%).

Probabilistic programming and deep learning The *coin-ball* problem is a standard example in the probabilistic programming community [De Raedt and Kimmig, 2014]. It describes a game in which we have a potentially biased coin and two urns. The first urn contains a mixture of red and blue balls, and the second urn a mixture of red, blue and green balls. To play the game, we toss the coin and take a ball out of each urn. We win if both balls have the same colour, or if the coin came up heads and we have at least one red ball. We want to learn the bias of the coin (the probability of heads), and the ratio of the coloured balls in each urn. We simultaneously train one neural network to classify an image of the coin as being heads or tails (`coin/2`), and a neural network to classify the colour of the ball as being either red, blue or green (`colour/4`). These are given as RGB triples. Task **T6** is thus to learn the `game/4` predicate, requiring a combination of subsymbolic reasoning, learning and probabilistic reasoning. The input consists of an image, two RGB pairs and the output is the outcome of the game. The coin-ball problem uses a very simple neural network component. Training on a set of 256 instances converges after 5 epochs, leading to 100% accuracy on the test set (64 instances). At this point, both networks correctly classify the colours and the coins, and the probabilistic parameters reflect the distributions in the training set.

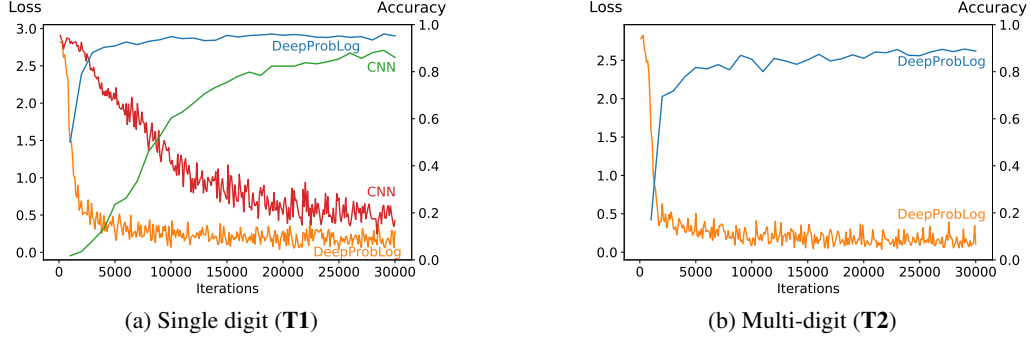


Figure 3: MNIST Addition problems: displaying training loss (red for CNN, orange for DeepProbLog) and F1 score on the test set (green for CNN, blue for DeepProbLog).

Test Length		Sorting (T4): Training length					Addition (T3): training length		
		2	3	4	5	6	2	4	8
$\partial 4$ [Bošnjak et al., 2017]	8	100.0	100.0	49.22	–	–	100.0	100.0	100.0
	64	100.0	100.0	20.65	–	–	100.0	100.0	100.0
DeepProbLog	8	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
	64	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0

(a) Accuracy on the addition (T3) and sorting (T4) problems (results for $\partial 4$ reported by Bošnjak et al. [2017]).

Training length $\rightarrow$	2	3	4	5	6
$\partial 4$ on GPU	42 s	160 s	–	–	–
$\partial 4$ on CPU	61 s	390 s	–	–	–
DeepProbLog on CPU	11 s	14 s	32 s	114 s	245 s

(b) Time until 100% accurate on test length 8 for the sorting (T4) problem.

Table 1: Results on the Differentiable Forth experiments

Implementation details In all experiments we optimize the cross-entropy loss between the predicted and desired query probabilities. The network used to classify MNIST images is a basic architecture based on the PyTorch tutorial. It consists of 2 convolutional layers with kernel size 5, and respectively 6 and 16 filters, each followed by a maxpool layer of size 2, stride 2. After this come 3 fully connected layers of sizes 120, 84 and 10 (19 for the CNN baseline). It has a total of 44k parameters. The last layer is followed by a softmax layer, all others are followed by a ReLu layer. The colour network consists of a single fully connected layer of size 3. For all experiments we use Adam [Kingma and Ba, 2015] optimization for the neural networks, and SGD for the logic parameters. The learning rate is 0.001 for the MNIST network, and 1 for the colour network. For robustness in optimization, we use a warm-up of the learning rate of the logic parameters for the coin-ball experiments, starting at 0.0001 and raising it linearly to 0.01 over four epochs. For the Forth experiments, the architecture of the neural networks and other hyper-parameters are as described in Bošnjak et al. [2017]. For the Coin-Urn experiment, we generate the RGB pairs by adding Gaussian noise ($\sigma = 0.03$) to the base colours in the HSV domain. The coins are MNIST images, where we use even numbers as heads, and odd for tails. For the implementation we integrated ProbLog2 [Dries et al., 2015] with PyTorch [Paszke et al., 2017]. We do not perform actual mini-batching, but instead use gradient accumulation. All programs are listed in the appendix.

7 Related Work

Most of the work on combining neural networks and logical reasoning comes from the *neuro-symbolic reasoning* literature [Garcez et al., 2012, Hammer and Hitzler, 2007]. These approaches typically focus on approximating logical reasoning with neural networks by encoding logical terms in Euclidean space. However, they neither support probabilistic reasoning nor perception, and are often limited to non-recursive and acyclic logic programs [Hölldobler et al., 1999]. DeepProbLog takes a different approach and integrates neural networks into a probabilistic logic framework, retaining the full power of both logical and probabilistic reasoning and deep learning.

The most prominent recent line of related work focuses on developing differentiable frameworks for logical reasoning. Rocktäschel and Riedel (2017) introduce a differentiable framework for theorem proving. They re-implemented Prolog’s theorem proving procedure in a differentiable manner and enhanced it with learning subsymbolic representation of the existing symbols, which are used to handle noise in data. Whereas Rocktäschel and Riedel (2017) use logic only to construct a neural network and focus on learning subsymbolic representations, DeepProbLog focuses on tight interactions between the two and parameter learning for both the neural and the logic components. In this way, DeepProbLog retains the best of both worlds. While the approach of Rocktäschel and Riedel could in principle be applied to tasks T1 and T5, the other tasks seem to be out of scope. Cohen et al. (2018) introduce a framework to compile a tractable subset of logic programs into differentiable functions and to execute it with neural networks. It provides an alternative probabilistic logic but it has a different and less developed semantics. Furthermore, to the best of our knowledge it has not been applied to the kind of tasks tackled in the present paper. The approach most similar to ours is that of Bošnjak et al. [2017], where neural networks are used to fill in *holes* in a partially defined Forth program. DeepProbLog differs in that it uses ProbLog as the host language which results in native support for both logical and probabilistic reasoning, something that has to be manually implemented in differentiable Forth. Differentiable Forth has been applied to tasks T3-5, but it is unclear whether it could be applied to the remaining ones. Finally, Evans and Grefenstette (2018) introduce a differentiable framework for rule induction, that does not focus on the integration of the two approaches like DeepProbLog.

A different line of work centers around including background knowledge as a regularizer during training. Diligenti et al. [2017] and Donadello et al. [2017] use FOL to specify constraints on the output of the neural network. They use fuzzy logic to create a differentiable way of measuring how much the output of the neural networks violates these constraints. This is then added as an additional loss term that acts as a regularizer. More recent work by Xu et al. [2018] introduces a similar method that uses probabilistic logic instead of fuzzy logic, and is thus more similar to DeepProbLog. They also compile the formulas to an SDD for efficiency. However, whereas DeepProbLog can be used to specify probabilistic logic programs, these methods allow you to specify FOL constraints instead.

Dai et al. [2018] show a different way to combine perception with reasoning. Just as in DeepProbLog, they combine domain knowledge specified as purely logical Prolog rules with the output of neural networks. The main difference is that DeepProbLog deals with the uncertainty of the neural network’s output with probabilistic reasoning, while Dai et al. do this by revising the hypothesis, iteratively replacing the output of the neural network with anonymous variables until a consistent hypothesis can be formed.

An idea similar in spirit to ours is that of Andreas et al. (2016), who introduce a neural network for visual question answering composed out of smaller modules responsible for individual tasks, such as object detection. Whereas the composition of modules is determined by the linguistic structure of the questions, DeepProbLog uses logic programs to connect the neural networks. These successes have inspired a number of works developing (probabilistic) logic formulations of basic deep learning primitives [Šourek et al., 2018, Dumančić and Blockeel, 2017, Kazemi and Poole, 2018].

8 Conclusion

We introduced DeepProbLog, a framework where neural networks and probabilistic logic programming are integrated in a way that exploits the full expressiveness and strengths of both worlds and can be trained end-to-end based on examples. This was accomplished by extending an existing probabilistic logic programming language, ProbLog, with neural predicates. Learning is performed by using aProbLog to calculate the gradient of the loss which is then used in standard gradient-descent based methods to optimize parameters in both the probabilistic logic program and the neural networks. We evaluated our framework on experiments that demonstrate its capabilities in combined symbolic and subsymbolic reasoning, program induction, and probabilistic logic programming.

Acknowledgements

RM is a SB PhD fellow at FWO (1S61718N). SD is supported by the Research Fund KU Leuven (GOA/13/010) and Research Foundation - Flanders (G079416N)